System Design Document

<u>Urban Smart Parking Management System</u>

(Team -1)

Farzan Faisal - B240179CS

Elias Joby - B240175CS

Reja Fairooz - B240281CS

Amil K - B240136CS

Ronn Boban - B240286CS

Kona Bhuvaneswar Reddy - B240214CS

DBMS Project

Class : CS02

Date: March 24, 2026

Table of Contents

1. Introduction
2. Purpose
3. System Overview
4. Design Constraints
5. System Architecture
6. Database Design
7. Technical Architecture Overview
8. System Security and Integrity Controls
9. Project References
10. Glossary

Introduction

This System Design Document has been created to outline the proposed system design for the new Urban Smart Parking Management System. The system is intended to provide a comprehensive, web-based solution to streamline the administration and experience of city parking. By designing, testing, and deploying this system, the management of parking infrastructure, session tracking, and user financial administration will be greatly improved. This document and its technical specifications comply with all required technical standards and infrastructure for the DBMS project.

Purpose

The purpose of this System Design Document is to provide a description for how the Urban Smart Parking Management System will be constructed. The Systems Design Document was created to ensure that the design meets the requirements specified for the project. The System Design Document provides a description of the system architecture, software, hardware, database design, and security.

System Overview

Historically, managing urban parking spaces, tracking fleet metrics, and enforcing parking dues has been challenging. The proposed Urban Smart Parking Management System tool will utilize a robust tech stack to provide an enterprise web tool which will standardize and improve the efficiency of maintenance and parking management capabilities.

The system is designed as an enterprise software tool functioning on a Client-Server Architecture utilizing RESTful APIs. It is compliant with standard web server deployment protocols and utilizes a relational database for backend continuity.

The system provides various user interfaces which allow data entry, updates, tracking, and report generation for both Admins and Drivers.

The new Urban Smart Parking Management System will provide the following capabilities:

- Real-time discovery of parking lots and automated calculation of session availability.
- Integration of all maintenance data which allows for real-time reporting of active vehicles, slots, and total dues.
- Enhanced user interfaces customized to the logged-in role (Admin vs. Driver).
- Simplified transactions for reserving and releasing slots.

Design Constraints

The Project Team identified several constraints which will impact and limit the design of the tool:

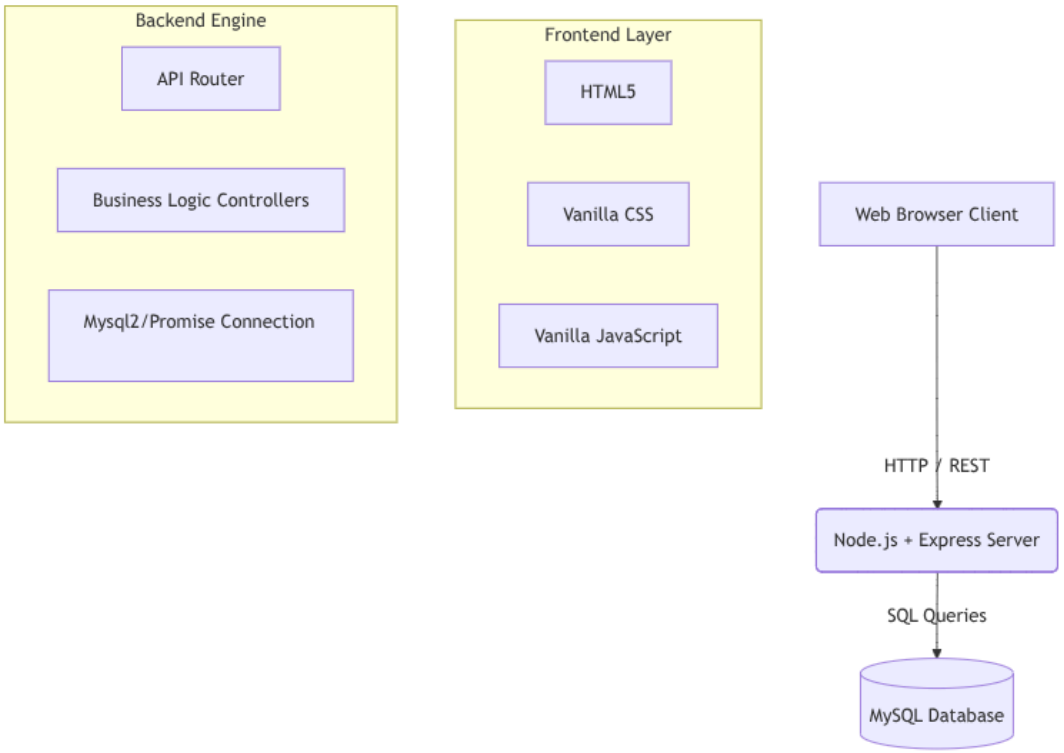
- **Environment Constraints**: The system must operate efficiently within local environments and cloud deployments (e.g. Render) using standard Node.js infrastructure.
- **Relational Database Management**: The database schema must be strictly relational (MySQL) to fulfill DBMS project requirements, requiring careful concurrency management for active parking sessions.
- **Dependency Minimization**: The frontend must rely entirely on HTML, CSS, and JS to maintain lightweight performance without framework overhead.

System Architecture

Hardware: The system is designed to be hosted on standard Virtual Machines or Cloud PaaS (Platform as a Service) instances, such as a Render Web Service for the backend node.js application and a managed MySQL database instance for data persistence.

Software: The software architecture follows a Client-Server Monolith model. The components which comprise the software architecture include:

- **Frontend Layer**: Vanilla HTML5, CSS3, and JavaScript interacting via standardized HTTP `fetch` APIs.
- **Backend / API Router**: Express.js server providing routing for business logic controllers.
- **Database Engine**: MySQL Connection pool utilized via the `mysql2/promise` library to securely inject and fetch structured statements.



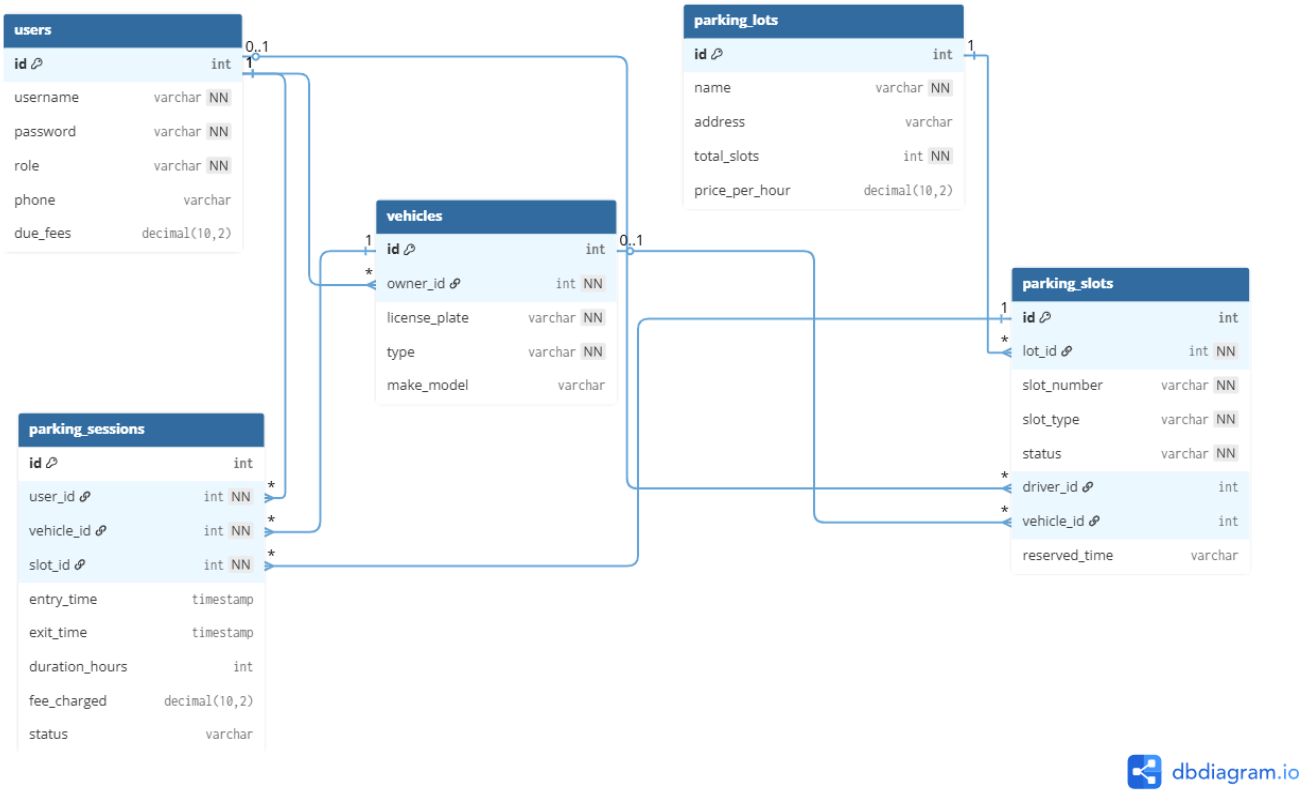
Database Design

The database is built on MySQL, ensuring ACID (Atomicity, Consistency, Isolation and Durability) compliance and referential integrity across all entities.

Structured data stored in the database is fully normalized. The major entities include `users`, `vehicles`, `parking_lots`, `parking_slots`, and `parking_sessions`.

Additional fields have been engineered to capture specific financial and categorical points:

- `price_per_hour` on parking lots to drive the algorithm.
- `due_fees` tracking user debt dynamically.
- `fee_charged` stamped immutably onto completed sessions.
- `Status` enumerations for slot concurrency control (`available`, `occupied`, `reserved`).



Technical Architecture Overview

User Scoping & Security

The system is architected upon Role-Based Access Control (RBAC), ensuring that user interactions are strictly limited to the functionalities associated with their specific roles. Access to the

internal API is secured by an Authentication Gateway which validates each request through JWT or Session-based token verification.

Administrative Tools

The administrative interface manages the system's core infrastructure and facilitates financial overrides:

- **Infrastructure Management:** Programmatic generation of parking assets, specifically defining parking lots and individual slots.
- **Manual Adjustments:** Allows admins to manually fix billing errors and override driver dues to resolve payment discrepancies and set customized dues.

Parking Logic & Driver Discovery

This module regulates the fundamental business logic of the application, utilizing Atomic MySQL Transactions to manage the reservation sequence. This technical implementation prevents race conditions, ensuring that a `parking_slot` is uniquely locked to a single driver and vehicle at any given moment.

Automated Billing

The billing engine triggers as soon as a slot is released. It calculates the total session duration, computes the cost, and automatically updates the `due_fees` on the driver's account.

System Security and Integrity Controls

We've built several security layers into the system to keep all the data safe and running smoothly:

- **User Permissions (RBAC):** Our system checks every request to make sure the person is either an `admin` or a `driver`. This keeps people from doing things they aren't supposed to do in the app.

- **Database Concurrency Protection:** The `PARKING_SLOTS` table acts as the system of record for concurrency. Since each slot can only be linked to one `vehicle_id` or `driver_id` at a time, the database naturally prevents double-booking. By checking the `status` column before allowing a new assignment, we ensure that a spot cannot be claimed unless it is explicitly marked as available
- **Data Referential Integrity (Cascade Termination):** System constraints are managed via physical database cascade behaviors (i.e. deleting a lot forcibly evicts all related slots).
- **Environment Variable Protection:** Sensitive credentials required for Database URL connections are maintained through .env configurations that are injected strictly at runtime and never committed to source control.

Project References

- System Design Document Template - ProjectManagementDocs.com
- System Design Document Template - CMS
- Navathe Slides from Class

Glossary

- **API** - Application programming interface
- **PaaS** - Platform as a Service
- **ACID** - Atomicity, Consistency, Isolation and Durability
- **RBAC** - Role-Based Access Control
- **JWT** - JSON Web Token